

ELEC6910J

Deep Reinforcement Learning

Lecture 9. Monte-Carlo Prediction

Course Instructor: Ling PAN

Department of Electronic and Computer Engineering

Hong Kong University of Science and Technology

Most slides are adapted from UC Berkeley CS188/CS285, CMU 10-403

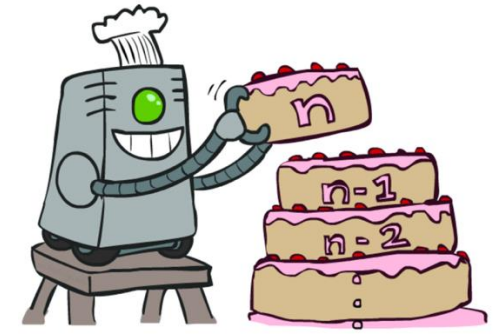
Recap: Value Iteration

- Bellman equation **characterizes** the optimal values

$$V^*(s) = \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V^*(s')]$$

- Value iteration **computes** the optimal values

$$V_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V_k(s')]$$



Recursion → Iteration

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$ 
|   Loop for each  $s \in \mathcal{S}$ :
|      $v \leftarrow V(s)$ 
|      $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$ 
|      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
until  $\Delta < \theta$ 
```

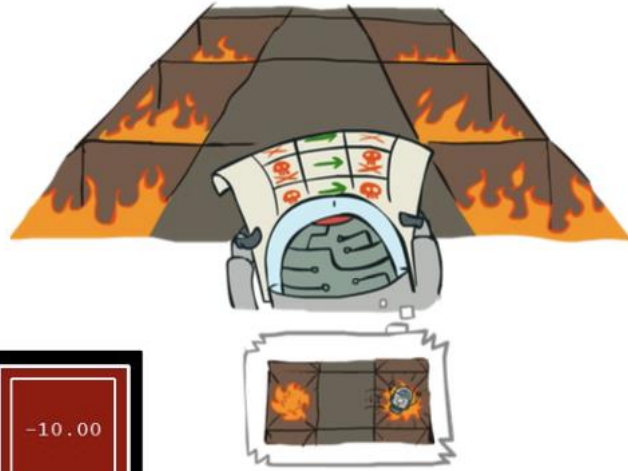
Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$$

- The policy often converges long before the values
→ What about directly learning the policy?

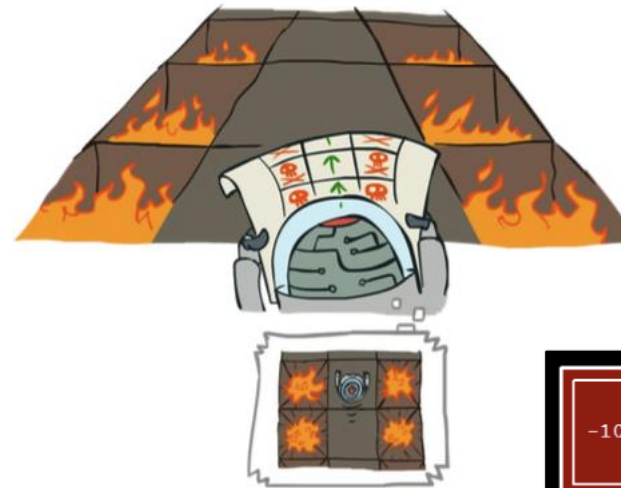
Recap: Policy Evaluation

Always Go Right



-10.00	100.00	-10.00
-10.00	1.09 ▶	-10.00
-10.00	-7.88 ▶	-10.00
-10.00	-8.69 ▶	-10.00

Always Go Forward



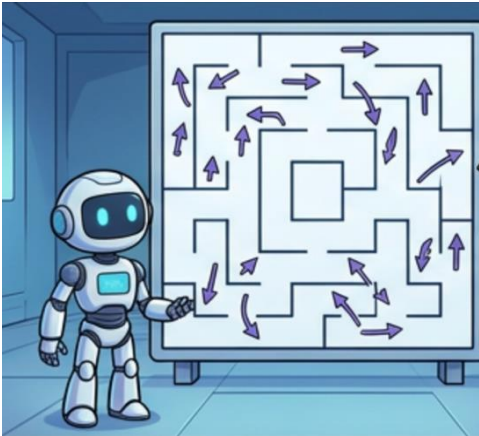
-10.00	100.00	-10.00
-10.00	70.20 ▲	-10.00
-10.00	48.74 ▲	-10.00
-10.00	33.30 ▲	-10.00

Usage: computing the value function for a fixed policy is to help find better policies.

Recap: Policy Evaluation

- How do we calculate the value function for a fixed policy π ?
 - Turn recursive Bellman expectation equations into updates (like value iteration)
→ Iterative Policy Evaluation

Input: a given policy π



Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\text{terminal})$ to 0

Loop:

$\Delta \leftarrow 0$

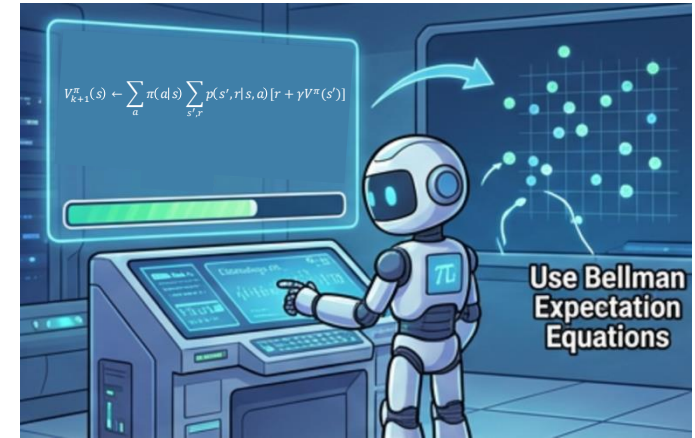
Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

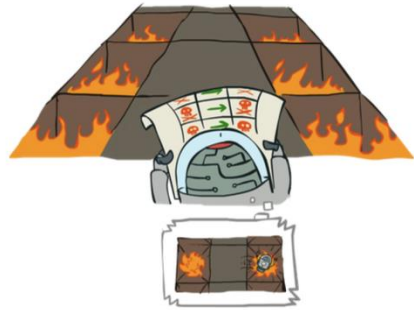
until $\Delta < \theta$



Recap: Policy Improvement

- Given a policy and its value function, we can easily evaluate a change in the policy at a single state.

Policy π : Always Go Right



Value function V^π

-10.00	100.00	-10.00
-10.00	1.09	-10.00
-10.00	-7.88	-10.00
-10.00	-8.69	-10.00

The core idea: testing a new action



New policy:

- Go forward
- Follow π (always go right) thereafter

- Key criterion: $Q^\pi(s, a) \begin{matrix} \searrow \\ \swarrow \end{matrix} V^\pi(s) \quad ???$
 - If $Q^\pi(s, a) > V^\pi(s)$:
 - Better to select a every time s is encountered
 - The new policy would in fact be a better one overall

Recap: Policy Iteration

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) \doteq 0$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

$\text{policy-stable} \leftarrow \text{true}$

For each $s \in \mathcal{S}$:

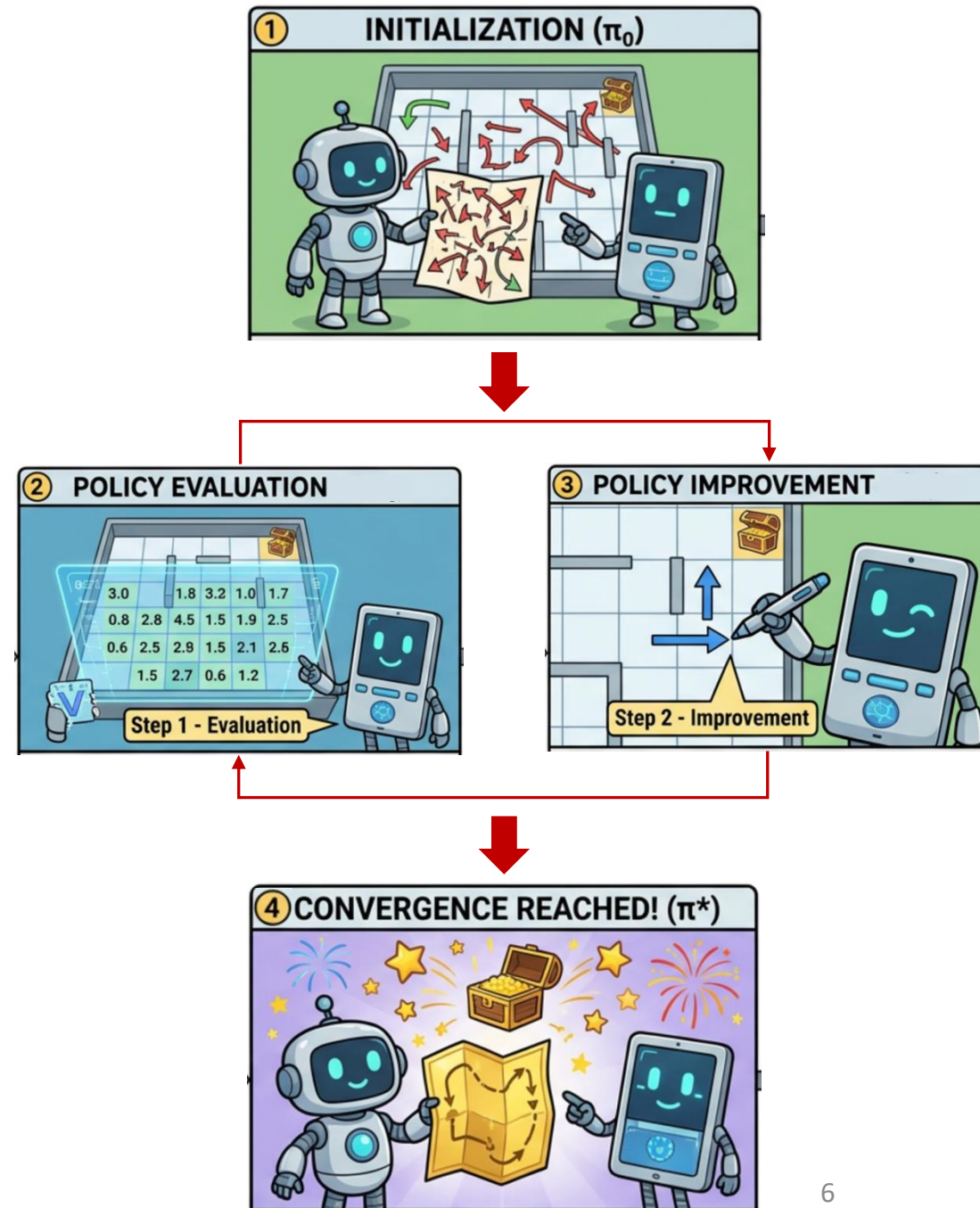
$\text{old-action} \leftarrow \pi(s)$

$\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If $\text{old-action} \neq \pi(s)$, then $\text{policy-stable} \leftarrow \text{false}$

If policy-stable , then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

[Demo](#)



Recap: Problems of DP

- So far, to estimate value functions we have been using dynamic programming with **known rewards and dynamics functions**:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]$$

- Next: estimate value functions and policies from interaction experience, **without** known rewards or dynamics.
- How? By sampling all the way. Instead of probabilities distributions to compute expectations, we will use empirical expectations by averaging sampled returns.

Monte-Carlo Prediction

- Motivation: the mystery world

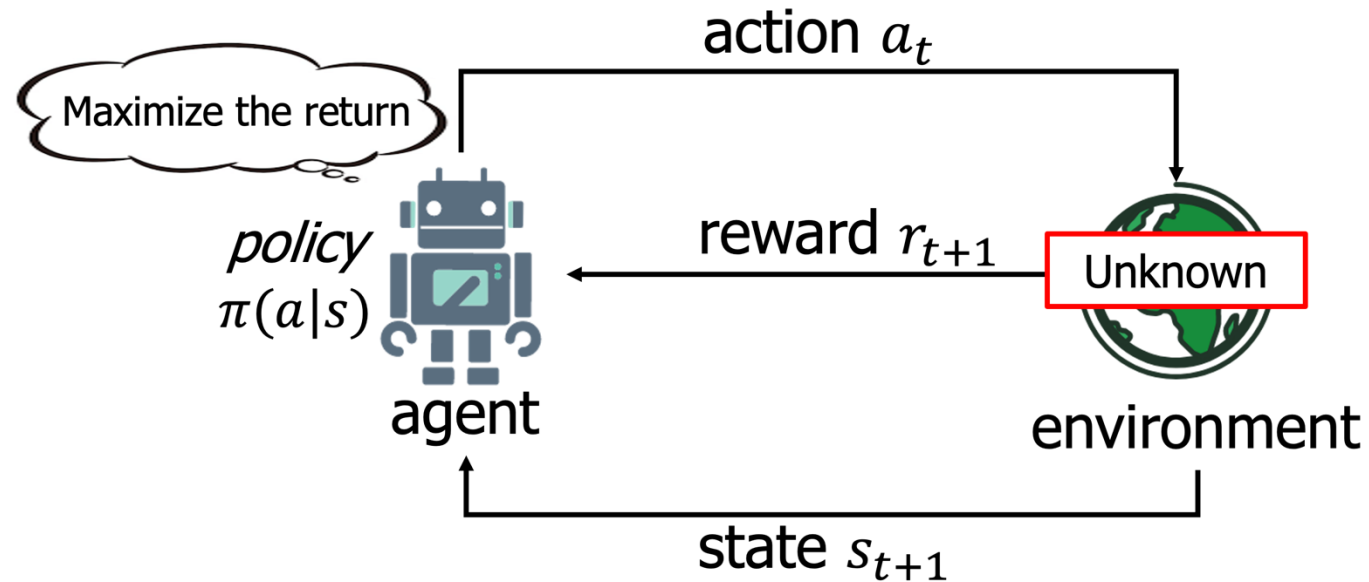
GREATEST UNSOLVED MYSTERIES



What is
 $P(s'|s, a)$ and $R(s, a, s')$?

Monte-Carlo Prediction

- Goal: **learn** $V^\pi(s)$ from episodes of experience under policy π



- We can get samples of trajectories $\{s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, r_{T-1}, s_T\} \sim \pi$ after running some policy π in the unknown world

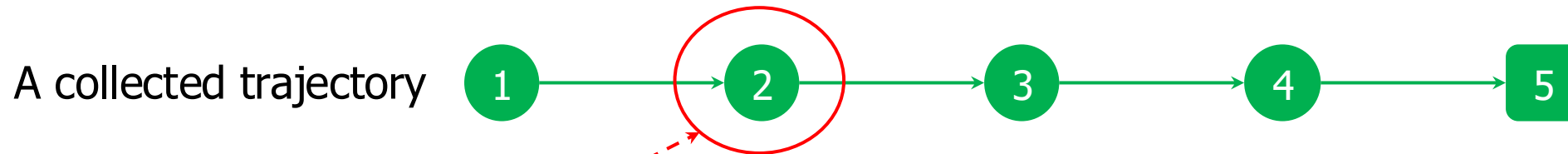
Monte-Carlo Prediction

- Goal: **learn** $V^\pi(s)$ from episodes of experience under policy π
- Idea: estimate it from experiences obtained by following π passing through s
 - average together observed sample values
- Act according to π
- Every time you visit a state s , write down what the sum of discounted rewards turn out to be
- Simply average the returns observed after visits to the state s
- Guarantee: as more returns are observed, the average should converge to the expected value



Monte-Carlo Prediction

- Goal: **learn** $V^\pi(s)$ from episodes of experience under policy π
- Method: average returns observed after visits to s



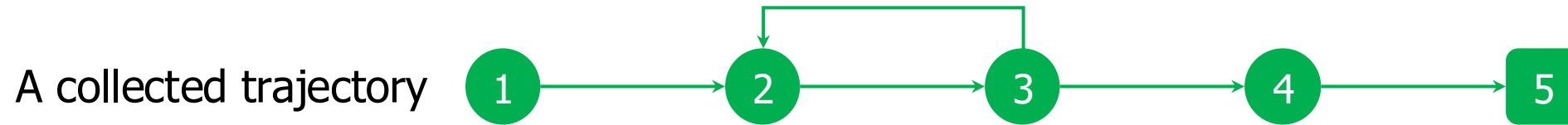
- Return (total discounted reward):
$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$
- Value function for policy π (expected return):
$$\underbrace{V^\pi(s)}_{\text{value of a state}} = \underbrace{\mathbb{E}_\pi}_{\text{expected}}[\underbrace{G_t}_{\text{discounted return}} | S_t = s] \text{ starting at state } s$$
- Monte-Carlo policy evaluation uses **empirical mean** return instead of expected return



What if a state is visited multiple times in the same episode?

Monte-Carlo Prediction

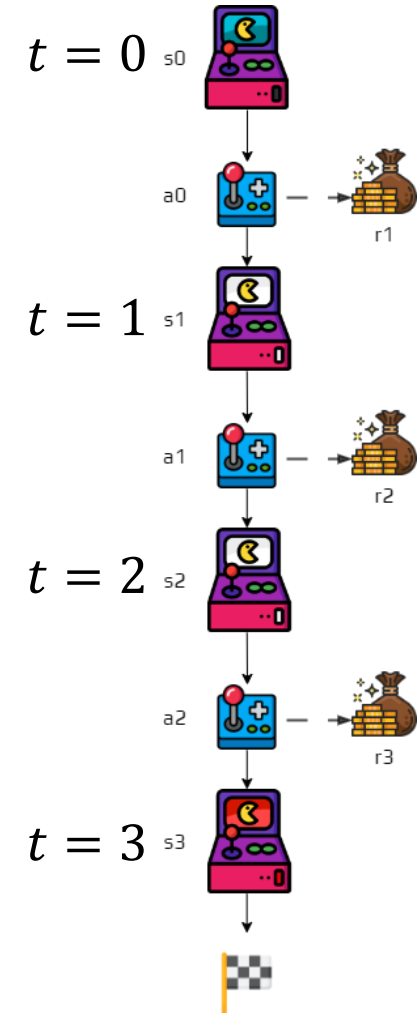
- Goal: **learn** $V^\pi(s)$ from episodes of experience under policy π
- Method: average returns observed after visits to s



- First-visit MC: average returns only for first time s is visited in an episode
- Every-Visit MC: average returns for every time s is visited in an episode

Monte-Carlo Prediction

- Example: calculate the return for state  using both the first and every visit.



First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

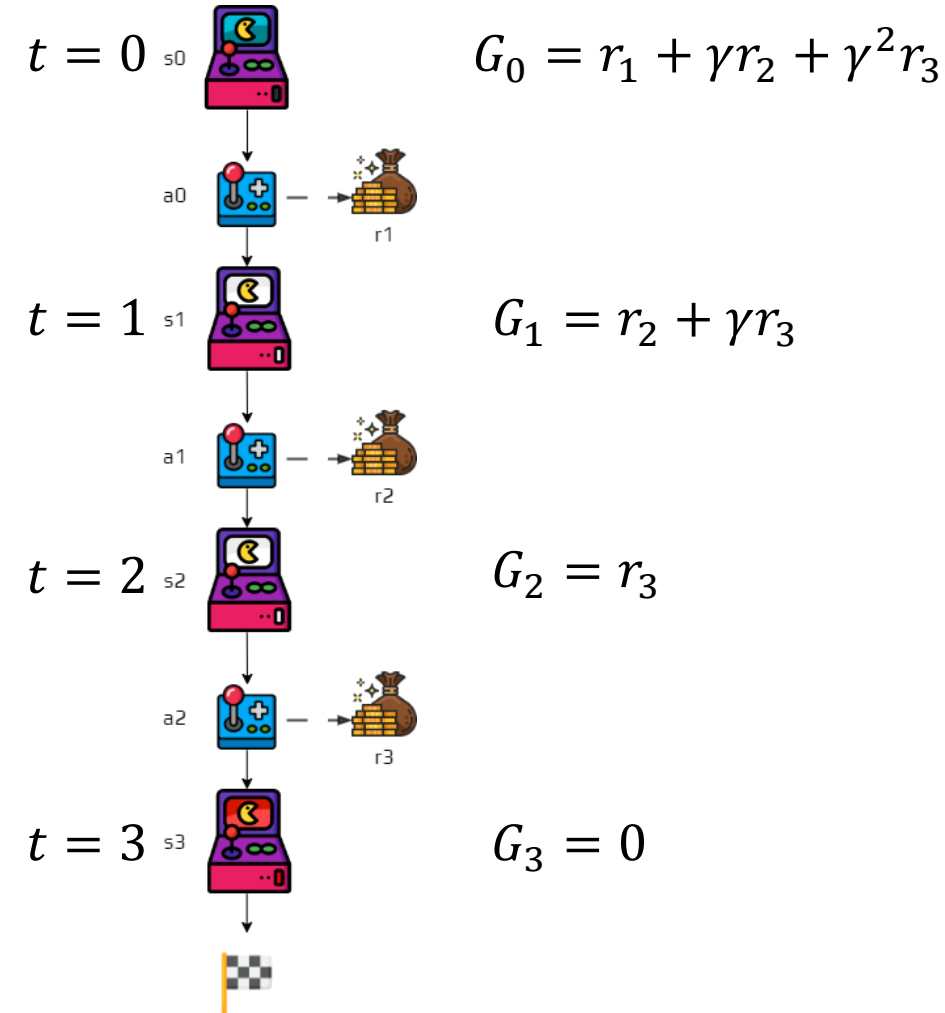
Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Monte-Carlo Prediction

- Example: calculate the return for state  using both the first and every visit.



First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

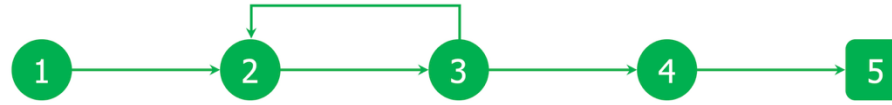
Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

- First-visit: $[G_1] \rightarrow V(s) = G_1 = r_2 + \gamma r_3$
- Every-visit: $[G_1, G_2] \rightarrow V(s) = \frac{G_1 + G_2}{2} = \frac{r_2 + 2\gamma r_3}{2}$

Monte-Carlo Prediction

- Algorithm: A collected trajectory



First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

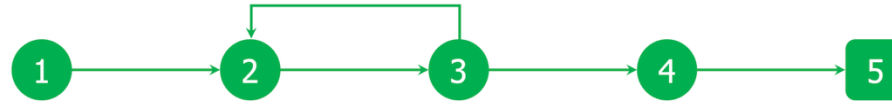
Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

- Note: Every-visit MC is the same except for the check for S_t having occurred earlier in the episode

Monte-Carlo Prediction

- Algorithm: A collected trajectory



First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \dots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Derivation: incremental mean

$$\begin{aligned}\mu_k &= \frac{1}{k} \sum_{j=1}^k x_j \\ &= \frac{1}{k} \left(x_k + \sum_{j=1}^{k-1} x_j \right) \\ &= \frac{1}{k} (x_k + (k-1)\mu_{k-1}) \\ &= \mu_{k-1} + \frac{1}{k} (x_k - \mu_{k-1})\end{aligned}$$

More efficient implementation: update $V(S_t)$ incrementally:

- $N(S_t) \leftarrow N(S_t) + 1$ (counter)
- $V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)} (G_t - V(S_t))$

For non-stationary problems, useful to track a running mean (forget older episodes):

- $V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$

Monte-Carlo Prediction

- Analysis

- Both converge to $V^\pi(s)$ as the number of visits (or first visits) to s goes to infinity.

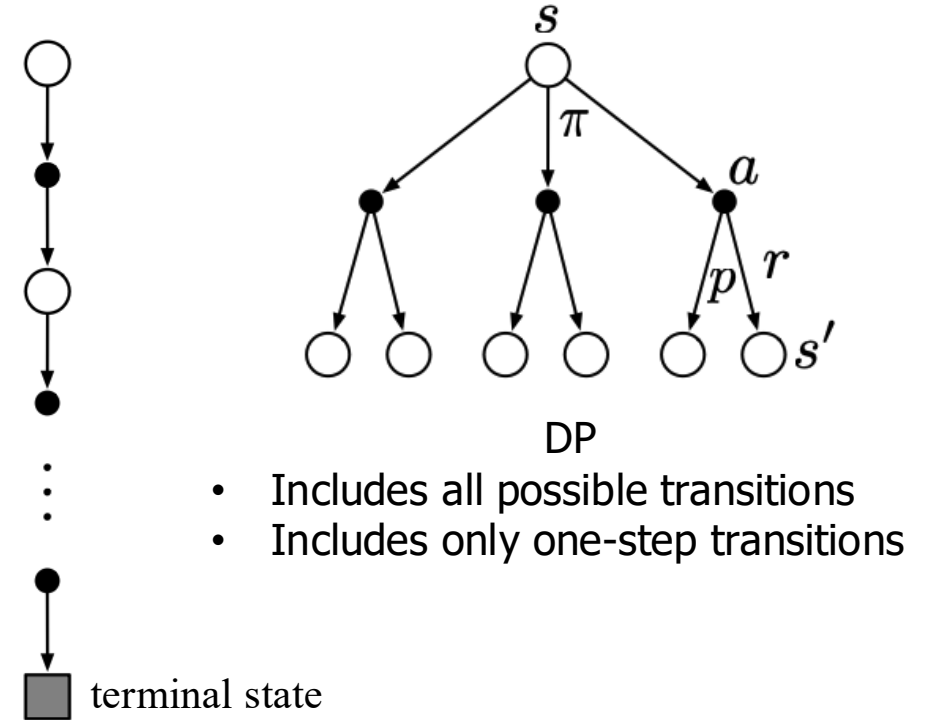
$$V(s) \rightarrow V^\pi(s) \text{ as } N(s) \rightarrow \infty$$

- Example: First-visit MC

- Each return is an independent, identically distributed estimate of $V^\pi(s)$ with finite variance.
 - By *the law of large numbers*, the sequence of averages of these estimates converges to their expected value.
 - Each average is itself an unbiased estimate, and the std of its error falls as $\frac{1}{\sqrt{n}}$ (n is the # of returns averaged).

Monte-Carlo Prediction

- Backup diagram:
 - Entire trajectory of transitions along a particular single episode ending at the terminal state is included
 - Only one choice considered at each state
 - **Does not bootstrap** from successor state's values (unlike DP)
 - The value estimates of later states are not used to inform the values of nearby states
 - Value is estimated by mean return



Monte-Carlo Prediction

- Is MC policy evaluation (without bootstrapping) good or bad?
 - Good, why?
 - Only from experience:
 - Does not require knowledge of P and R
 - Eventually computes the correct average values, using just sample transitions
 - Simple
 - Computational expense of estimating the value of a single state is independent of the number of states
 - Bad, why?
 - All episodes must terminate
 - Does not exploit the Bellman equation:
 - Wastes information about state connections
 - Each state must be learned separately
 - Require a lot of samples

Monte-Carlo Estimation of Action Values

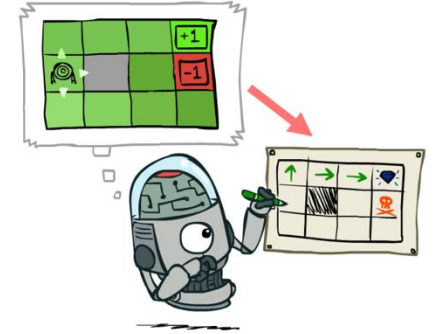
- Problem:

- **With** a model, state values alone are sufficient to determine a policy
 - Policy extraction: gets the policy implied by the values by doing a one-step lookup

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- **Without** a model, state values alone are **not sufficient**
 - Must explicitly estimate the value of each action in order for the values to be useful in suggesting a policy
 - Primary goal: estimate $Q^\pi(s, a)$ and $Q^*(s, a)$
- the case where MC is most useful

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_\pi [R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a') \right] \end{aligned}$$



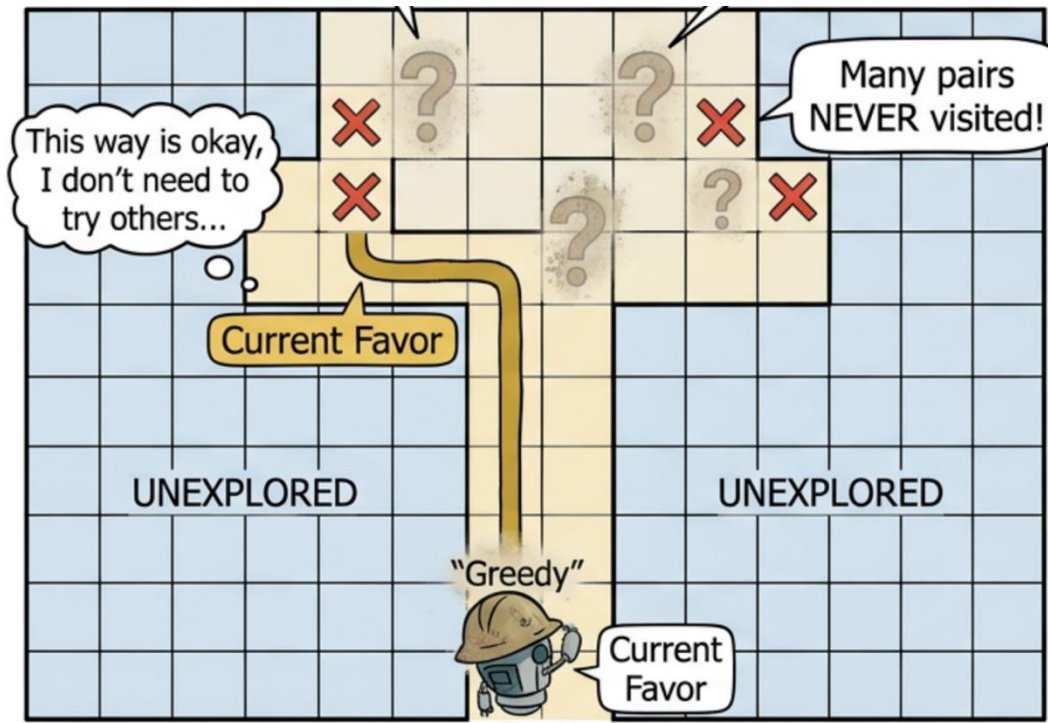
Monte-Carlo Estimation of Action Values

- Method: consider visits to a state-action pair rather than a state
 - First-visit MC
 - Every-visit MC

} → Converge quadratically to the true expected value as the number of visits to each state-action pair approaches infinity
- Convergence:
 - Also converge quadratically to the true expected values as the number of visits to each state-action pair approaches infinity

Monte-Carlo Estimation of Action Values

- More critical challenge: many state-action pairs may never be visited...
 - We need to compare alternatives and estimate the value of all the actions from each state later, not just the one we currently favor
- Maintain exploration



- Potential solution: exploring starts (ES)
 - Specifying that the episodes start in a state-action pair, and that every pair has a non-zero probability of being selected as the start.
 - Guarantees that all state-action pairs will be visited an infinite number of times in the limit of an infinite number of episodes.

Monte-Carlo Prediction

- Summary so far:
 - **Unknown** dynamics: **estimate (learn)** **value functions** and **optimal policies** using **Monte Carlo**
 - Monte Carlo **prediction**: estimate the value function of a given policy by
 - Deploying it
 - Collect episodes
 - Average their returns
 - Next: Monte Carlo **control** -- find optimal policies by interaction



How Monte-Carlo estimation can be used to approximate optimal policies?

ELEC6910J

Deep Reinforcement Learning

Lecture 10. Monte-Carlo Control

Course Instructor: Ling PAN

Department of Electronic and Computer Engineering

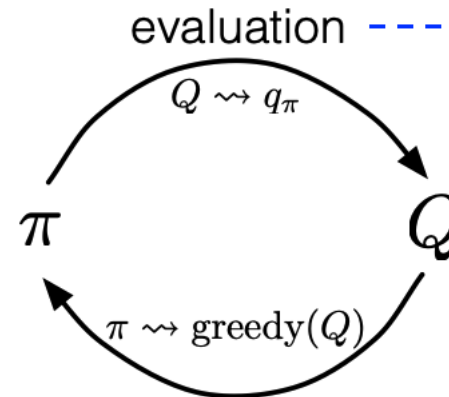
Hong Kong University of Science and Technology

Most slides are adapted from UC Berkeley CS188/CS285, CMU 10-403

Monte-Carlo Control

- Consider a Monte-Carlo version of classical policy iteration

$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*$$



MC policy iteration step:
Policy evaluation using MC methods
(followed by policy improvement)

Policy improvement step: $\leftarrow$ improvement
Greedy with respect to Q-function

$$\pi(s) = \operatorname{argmax}_a Q(s, a)$$

- Meaning: for each s , deterministically chooses an action with maximal Q-value
- Effect: construct each π_{k+1} as the greedy policy w.r.t. Q_k^π

$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \operatorname{argmax}_a q_{\pi_k}(s, a)) \\ &= \max_a q_{\pi_k}(s, a) \\ &\geq q_{\pi_k}(s, \pi_k(s)) \end{aligned}$$

Monte-Carlo Control with Exploring Starts

- Algorithm:

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
 $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
 $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0

Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$

Monte-Carlo Control with Exploring Starts

- Algorithm:

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
 $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
 $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0

Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$

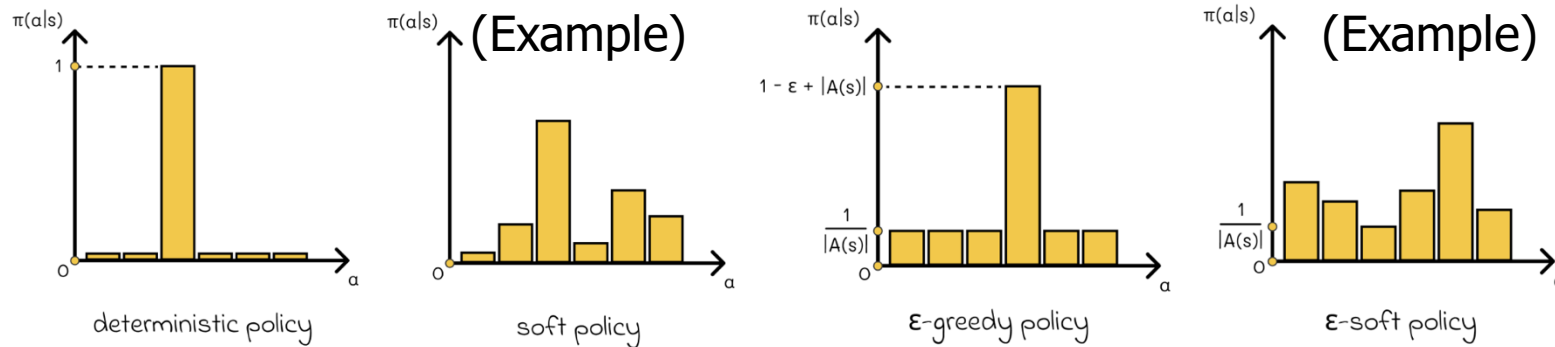
Monte-Carlo Control without Exploring Starts

- How can we avoid the unlikely assumption of exploring starts?
 - On-policy: evaluate or improve the policy that is used to make decisions
 - MC control with ES 
 - Every state–action pair can be selected as an initial pair
 - Exploration is provided by the starting mechanism
 - MC control with soft policies 
 - Let the policy itself maintain exploration
 - ...
 - Off-policy: evaluate or improve a policy different from that used to generate the data

Monte-Carlo Control with Soft Policies

- We will balance exploration-exploitation with **eternally soft policies**:

$$\pi(a|s) > 0 \text{ for all } s \text{ and } a$$



- Example: ϵ -greedy policy

- Non-greedy actions are given the minimal probability of selection $\frac{\epsilon}{|A(s)|}$
- Greedy actions are given the remaining bulk of the probability $1 - \epsilon + \frac{\epsilon}{|A(s)|}$

$$\pi(a|s) > \frac{\epsilon}{|A(s)|}$$

- More general class: ϵ -soft policy

$$\pi(a|s) \geq \frac{\epsilon}{|A(s)|} \text{ for all } s \text{ and } a \ (\epsilon > 0)$$

- Among ϵ -soft policies, ϵ -greedy policies are in some sense those that are closest to greedy

Monte-Carlo Control with Soft Policies

- Soft policy: $\pi(a|s) > 0$
- ϵ -soft policy: $\pi(a|s) > \frac{\epsilon}{|A(s)|}$
- ϵ -greedy policy with respect to Q:
 - Non-greedy actions are given the minimal probability of selection $\frac{\epsilon}{|A(s)|}$
 - Greedy actions are given the remaining part of the probability $1 - \epsilon + \frac{\epsilon}{|A(s)|}$

➤ Every ϵ -soft policy has two components

$$\pi(a | s) = \frac{\epsilon}{m_s} + (1 - \epsilon)\mu(a | s)$$

- ϵ : uniform exploration
 - $1 - \epsilon$: freely allocated action preference
- Example: for an ϵ -greedy policy
 - Allocate the entire remaining budget to maximizing actions

Monte-Carlo Control with Soft Policies

- Algorithm:

On-policy first-visit MC control (for ϵ -soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\epsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ϵ -soft policy

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

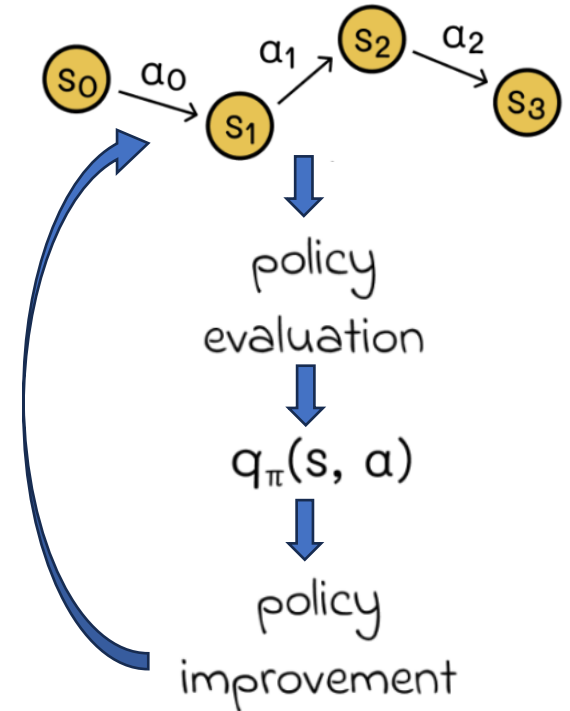
Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg \max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \epsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$



- Cannot simply improve the policy by making it greedy w.r.t. the current value function
→ Will prevent further exploration for non-greedy actions

→ Move it only to an ϵ -greedy policy

- Converges to the best ϵ -soft policy, but can eliminate the assumption of exploring starts

Monte-Carlo Control with Soft Policies

- Analysis: that any ϵ -greedy policy π' with respect to Q^π is an improvement over any ϵ -soft policy π is assured by the policy improvement theorem.

$$\begin{aligned} q_\pi(s, \pi'(s)) &= \sum_a \pi'(a|s) q_\pi(s, a) \\ &= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) + (1 - \epsilon) \max_a q_\pi(s, a) \\ &\geq \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) + (1 - \epsilon) \sum_a \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}(s)|}}{1 - \epsilon} q_\pi(s, a) \\ &= \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) - \frac{\epsilon}{|\mathcal{A}(s)|} \sum_a q_\pi(s, a) + \sum_a \pi(a|s) q_\pi(s, a) \\ &= v_\pi(s). \end{aligned}$$

- 
- The sum is a weighted average with non-negative weights summing to 1
 - The weighted sum must be less than or equal to the largest number

- Therefore, by the policy improvement theorem, $\pi' \geq \pi$ ($V^{\pi'}(s) \geq V^\pi(s)$ for all s)
- The proof of that the equality can hold only when both π' and π are optimal among the ϵ -soft policies are omitted here.

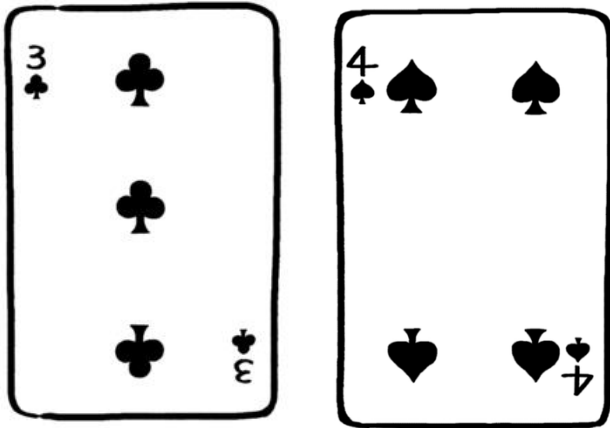
Monte-Carlo Control with Soft Policies

- Practical example: solving Blackjack
 - The object of the popular casino card game of blackjack is to obtain cards the sum of whose numerical values is as great as possible without exceeding 21.
 - All face cards count as 10.
 - An ace can either count as 1 or as 11.

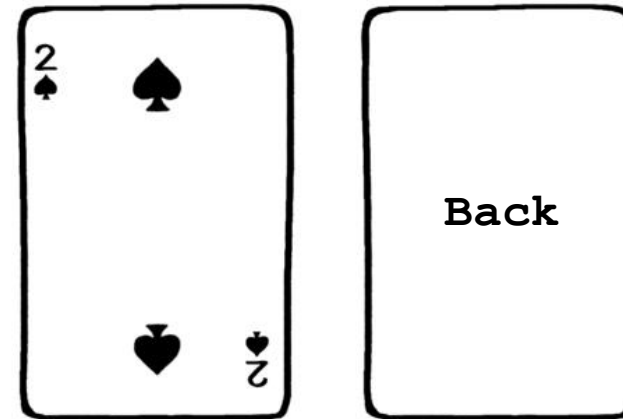


Monte-Carlo Control with Soft Policies

- Practical example: solving Blackjack
 - Each player competes independently against the dealer.
 - The game begins with two cards dealt to both dealer and player. One of the dealer's cards is face up and the other is face down.



Player



Dealer

Action:

- Hit: request additional cards, one by one
- Stick: stop → becomes the dealer's turn

If the sum exceeds 21, he goes bust → loses

If the player holds an ace that he could count as 11 without going bust, then the ace is said to be usable.

Policy: sticks on any sum of 17 or greater, and hits otherwise

Monte-Carlo Control with Soft Policies

- Practical example: solving Blackjack
 - Model: episodic finite MDP
 - Each game of blackjack is an episode
 - Rewards: +1 for winning, -1 for losing, 0 for drawing
 - Intermediate rewards are all 0
 - Do not discount ($\gamma = 1$)
 - Terminal rewards are also the returns

- The state can be simplified as a combination of

dealer's one showing card	(ace, 2-10)
---------------------------	-------------

his current sum	(12-21)
-----------------	---------

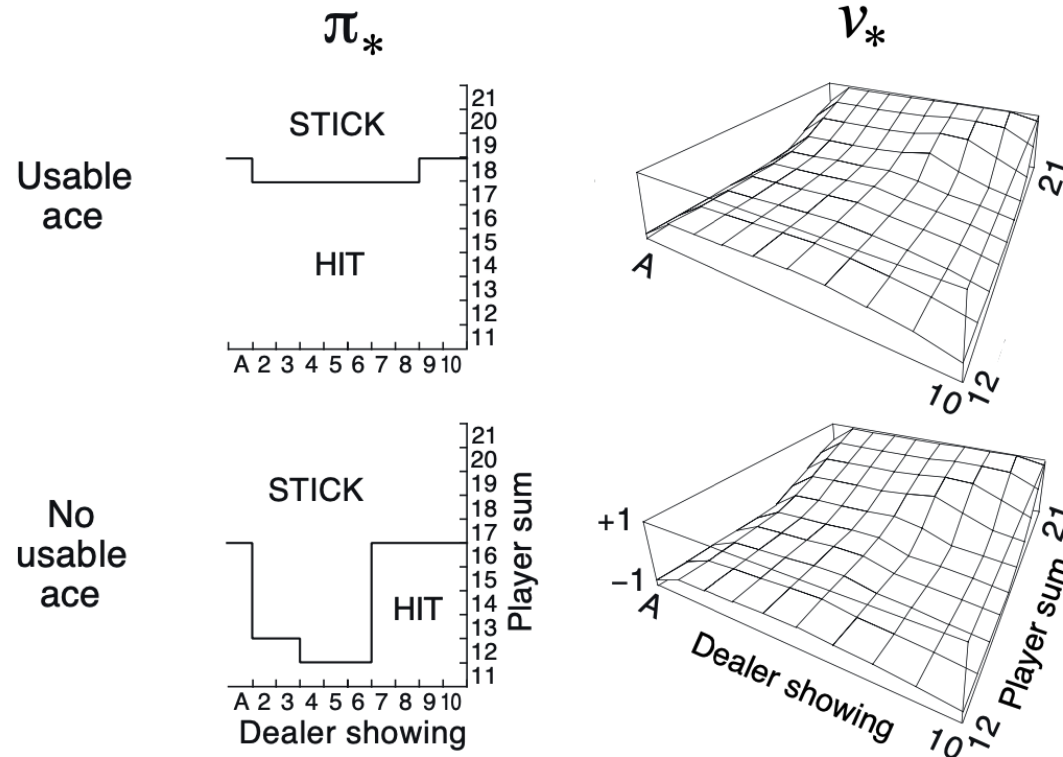
if he holds a usable ace	{True, False}
--------------------------	---------------

When your hand totals 11 or less, the strategy is clear: always hit, since you can't possibly bust.

200 possible states

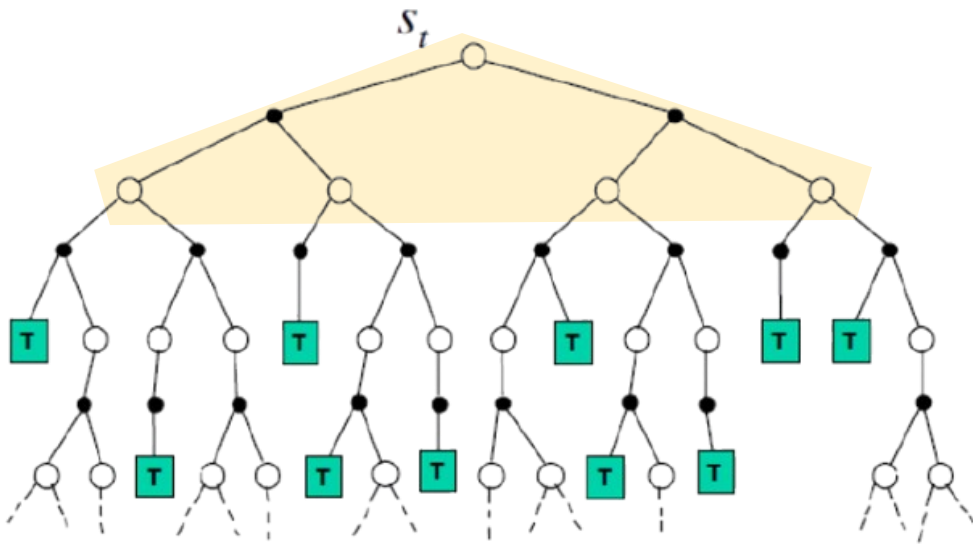
Monte-Carlo Control with Soft Policies

- Practical example: solving Blackjack
 - Consider the policy that sticks if the player's sum is 20 or 21, and otherwise hits.
 - To find the state-value function for this policy by a Monte Carlo approach, one simulates many blackjack games using the policy and averages the returns following each state.



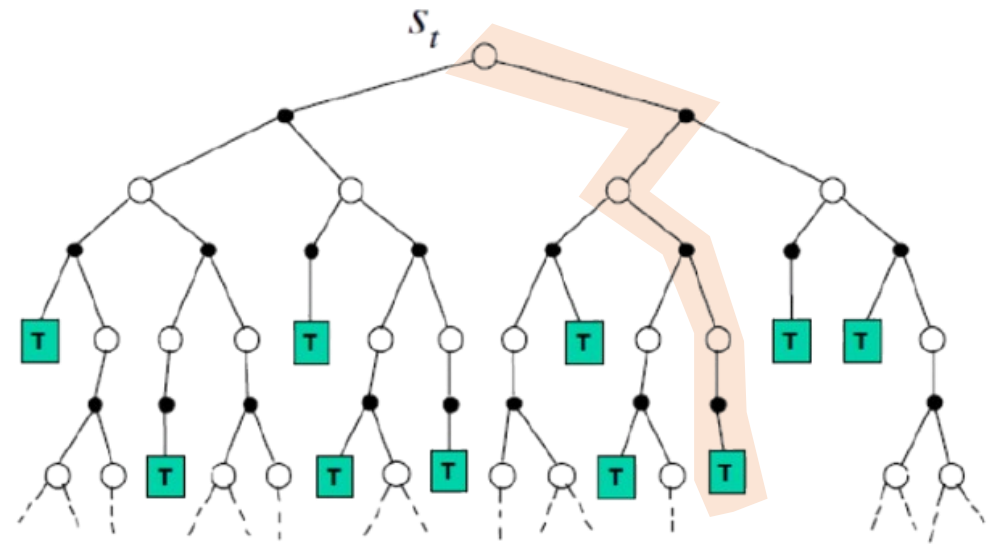
Dynamic Programming v.s. Monte Carlo

- Policy evaluation (prediction)
 - For a given policy π , compute the state-value function V^π



Dynamic Programming

$$\begin{aligned} V(S_t) &\leftarrow \mathbb{E}_\pi[R_{t+1} + \gamma V(S_{t+1})] \\ &= \sum_a \pi(a|S_t) \sum_{s', r} p(s', r|S_t, a) [r + \gamma V(s')] \end{aligned}$$



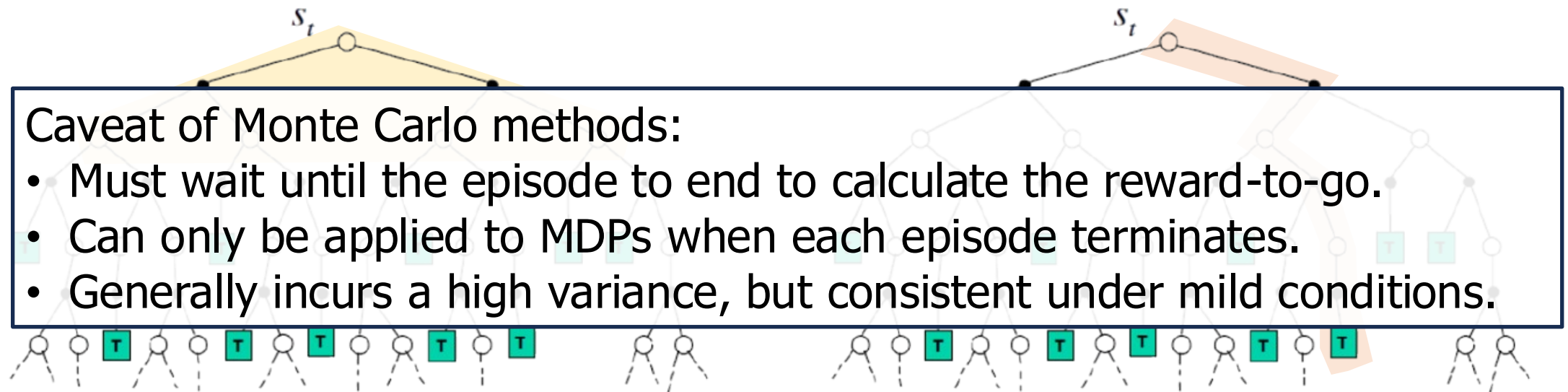
Monte-Carlo Methods

- Every-visit Monte-Carlo prediction:
$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

target: the actual return after time t

Dynamic Programming v.s. Monte Carlo

- Policy evaluation (prediction)
 - For a given policy π , compute the state-value function V^π



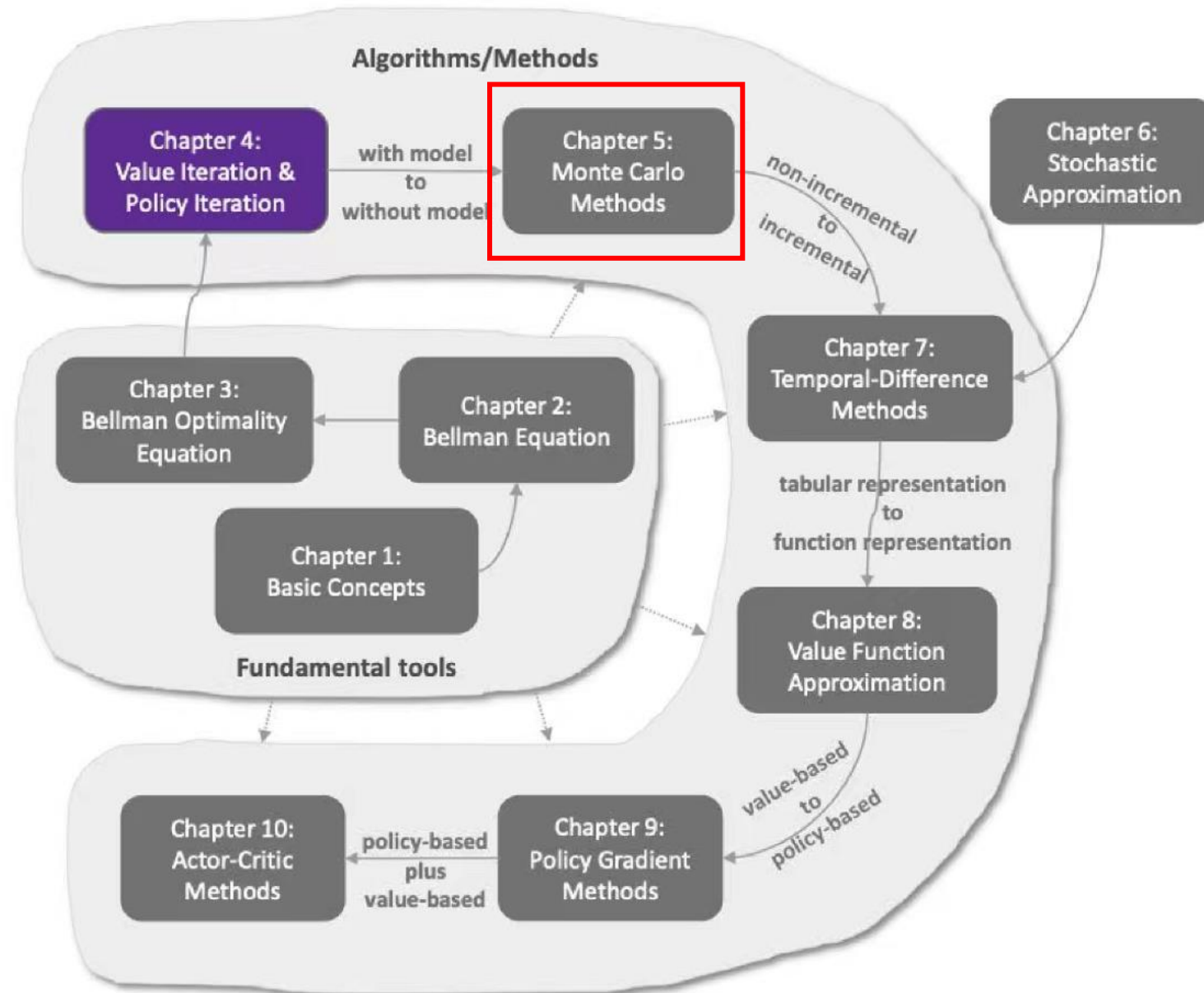
Dynamic Programming

$$\begin{aligned} V(S_t) &\leftarrow \mathbb{E}_\pi[R_{t+1} + \gamma V(S_{t+1})] \\ &= \sum_a \pi(a|S_t) \sum_{s', r} p(s', r|S_t, a)[r + \gamma V(s')] \end{aligned}$$

Monte-Carlo Methods

- Every-visit Monte-Carlo prediction:
 $V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$

target: the actual return after time t



Announcement

- Homework 1 has been posted on Canvas
 - Deadline: 11:59PM, Oct 16
 - Hints next week
- [Tutorial on setting up gym](#)

Thanks and Happy Holiday!